MBIST · BISR / BIRA · TEST ENGINEERING

RA Advisor

AI-assisted redundancy-analysis algorithm selection for memory built-in self-test — approach, reference implementation, and a production roadmap.

Every MBIST fail triggers a choice: which redundancy-analysis algorithm should the repair logic run to allocate spare rows and columns? Today that choice is often made by trying candidates in sequence — row, then column, then combinational — until one works, burning retest cycles on every die that isn't fixed by the first guess. This report documents an approach that predicts the right algorithm directly from the fail bitmap, and a working dashboard that demonstrates the full workflow end to end.

82%
TOP-1 ACCURACY, DEMO SESSION

20
RETEST CYCLES AVOIDED / 11 DIES

766ms
CUMULATIVE TIME SAVED

5
REPAIR CLASSES COVERED

PREPARED FOR
Krishna

DATE
August 29, 2026

REFERENCE BUILD
RA Advisor interactive dashboard

SCOPE
Technical deep-dive

01 Executive Summary

Memory built-in self-test (MBIST) locates faulty bits; built-in self-repair (BISR), driven by a built-in redundancy-analysis engine (BIRA), decides how to fix them by reallocating spare rows, columns, or block-level elements. The redundancy-analysis (RA) step has to choose among several repair algorithms — row-only, column-only, row+column combinational, or localized block redundancy — and a wrong or late choice is expensive twice over: it can waste spares that a later, correctly-chosen algorithm would have needed, and every extra algorithm the flow has to try before finding a working one is a full retest cycle on that die.

This report describes an AI-assisted alternative: predict the right RA algorithm directly from the fail bitmap, before any repair algorithm actually runs, so the legacy sequential trial-and-error flow becomes the fallback path rather than the default one. We cover the feature-engineering and model choices involved, a feasibility-aware way of generating training labels, and a working reference implementation — the **RA Advisor** dashboard — that simulates the whole loop end to end on synthetic fail bitmaps and reports the same accuracy and cycle-time metrics a production deployment would track. We close with a six-step roadmap for replacing the dashboard's illustrative scoring function with a model trained on real BISR/ATE repair logs.

02 Background: MBIST, BISR/BIRA, and the Retest Cost Problem

MBIST exercises an embedded memory with test algorithms (March-family patterns, checkerboard, walking 1/0, and similar) and captures a **fail bitmap** — a map of which bit cells failed. On its own, MBIST only diagnoses; repair is the job of BISR, and specifically its redundancy-analysis engine (BIRA), which decides how to spend a fixed budget of spare rows and spare columns (or, in modern designs, dedicated local block-level spare elements) to cover every failing cell.

The redundancy-analysis problem has a small number of well-known algorithm classes, summarized in Table 1.

CLASS	WHAT IT DOES	WORKS BEST WHEN…
Row redundancy	Swaps in spare rows for rows with clustered fails.	Fails concentrate in one or a few full rows and nowhere else.
Column redundancy	Swaps in spare columns.	Fails concentrate in one or a few full columns and nowhere else.
Row+column combinational	Uses both row and column spares together, solving a small covering problem.	Fails span both dimensions — the cheapest single-dimension fix can't cover everything.

CLASS	WHAT IT DOES	WORKS BEST WHEN…
Local block redundancy	Substitutes a small dedicated spare block for a compact, localized defect.	Fails cluster tightly in a small region (a process-induced localized defect) rather than along a full row or column.
No repair (ECC-covered)	Leaves the die as-is; on-die ECC corrects the residual errors at runtime.	Only one or two isolated bits fail — not worth spending any spare budget on.

The legacy way to pick among these is procedural: try the cheapest or most common algorithm first, check whether it fully repairs the die within the spare budget, and if not, try the next one. Each attempt is a real compute pass over the fail bitmap and, in many flows, a real retest of the die afterward to confirm the repair. When the correct algorithm happens to be third or fourth in that fixed try-order, the die pays for three or four passes instead of one — and at volume, across every die that needs a non-default repair class, that adds up to a measurable share of total test time.

> WHY THIS MATTERS AT VOLUME
>
> The cost of guessing wrong scales with die count, not with engineering effort — a fixed try-order that is occasionally wrong on 1000 dies costs 1000× the wasted passes. That is what makes RA algorithm selection a good target for a model trained on the pattern in the fail bitmap: it doesn't need to be perfect, only better on average than a fixed guess order.

03 Related Work

AI applied to semiconductor test has moved from research to production tooling over the last several years, along a few distinct fronts relevant to this problem:

- **Redundancy-analysis algorithm recommendation.** A deep learning model trained on fail-bitmap patterns to directly recommend which RA algorithm to run, rather than trying several in sequence — the approach this report builds on and demonstrates.[1]
- **AI-assisted failure analysis in DFT tooling.** EDA vendors now embed machine learning in their DFT/diagnosis flows to extract root-cause insight from scan and capture data — described by Siemens EDA as building a "digital twin" of failure analysis — with reported productivity gains from early adopters.[2]
- **Adaptive test and retest-skip decisions.** Gaussian Mixture Models, gradient-boosted trees, and conformal-prediction frameworks are used in production test programs to predict per-die pass/fail outcomes and skip tests when confidence is high, with GPU-accelerated deployments reporting roughly 25% test-time reduction while preserving defect coverage.[3]
- **Smarter, spatially-aware test limits.** Gaussian Process Regression and similar spatial models are replacing static Dynamic Part Average Testing (DPAT) so that normal wafer-level variation is no longer misclassified as a failure, reducing false retest triggers.[3]

- **Memory- and interface-specific BiST for advanced packaging.** For HBM and chiplet designs, dedicated BiST with loopback modes and lane-repair logic addresses defects that classical stuck-at fault models miss, and is an active area for AI-assisted analysis of repair outcomes.[4]

The RA Advisor concept in this report sits closest to the first item — it is a decision-support layer in front of the existing BISR/BIRA repair engine, not a replacement for MBIST or for the repair algorithms themselves.

04 Proposed AI Approach

4.1 Feature Engineering from Fail Bitmaps

The fail bitmap is the only input the model needs. Two feature strategies are viable:

- **Engineered tabular features** — row and column fail histograms, the count of rows/columns crossing a "mostly failed" threshold, the failing-cell density, and the bounding box and compactness of the failing region (a tight, small bounding box relative to the array indicates a localized defect; a bounding box spanning nearly the full width or height indicates a row- or column-shaped one). These feed cheaply into a gradient-boosted tree or a shallow classifier, with inference cost low enough to run inline in ATE software.
- **Raw bitmap as a 2-D image** — a small convolutional network trained directly on the `rows×cols` bitmap captures irregular spatial shapes (diagonal defects, multi-cluster patterns) that hand-engineered histograms can miss, at higher inference latency and model-maintenance cost.

Tabular features are the pragmatic default; a CNN is worth the added cost only once the fail population shows failure modes the tabular features demonstrably under-fit.

4.2 Model Architecture Options

Because the target is a small, fixed set of repair classes, this is a multiclass classification problem, not a regression or ranking problem. A gradient-boosted tree (XGBoost/LightGBM) over engineered features is the right starting point: fast to train, cheap to deploy, and its per-class scores double as the confidence figure an engineer needs to trust or override a recommendation. A CNN is the fallback for irregular spatial patterns, and a two-stage design — a fast tabular model that defers to a CNN only on bitmaps it is unsure about — is a reasonable middle ground once volume justifies the added complexity.

4.3 Feasibility-Aware Labeling

The most important, and easiest to get wrong, part of building the training set is the label itself. The correct label is not "whichever algorithm looks structurally right" — it is *whichever feasible algorithm both repairs the die and consumes the fewest spares*, given the die's actual spare-row and spare-column budget. Two failure modes follow directly from getting this wrong:

> LABELING PITFALL
>
> A bitmap that looks row-shaped is not automatically row-repairable: a single stray bit outside the dominant row means row-only redundancy leaves a residual fail, so the correct label is row+column combinational even though the pattern reads as "mostly a row." Any labeling pipeline that assigns ground truth from pattern shape alone, without simulating actual spare coverage, will systematically mislabel these borderline cases.

The reference implementation in Section 5 makes this feasibility check explicit and exhaustive — it is exactly the mechanism used to compute the "optimal" label the demo compares the model's recommendation against.

05 Reference Implementation: RA Advisor

RA Advisor is an interactive dashboard built to demonstrate the full decision loop — generate a fail bitmap, extract features, recommend an algorithm, check its feasibility against a spare budget, and compare the resulting repair time against the legacy sequential flow. It uses synthetic bitmaps and a transparent, tunable scoring function in place of a trained model, so every part of the pipeline is inspectable; Section 7 covers what changes to make it production-real.

5.1 Interface Overview

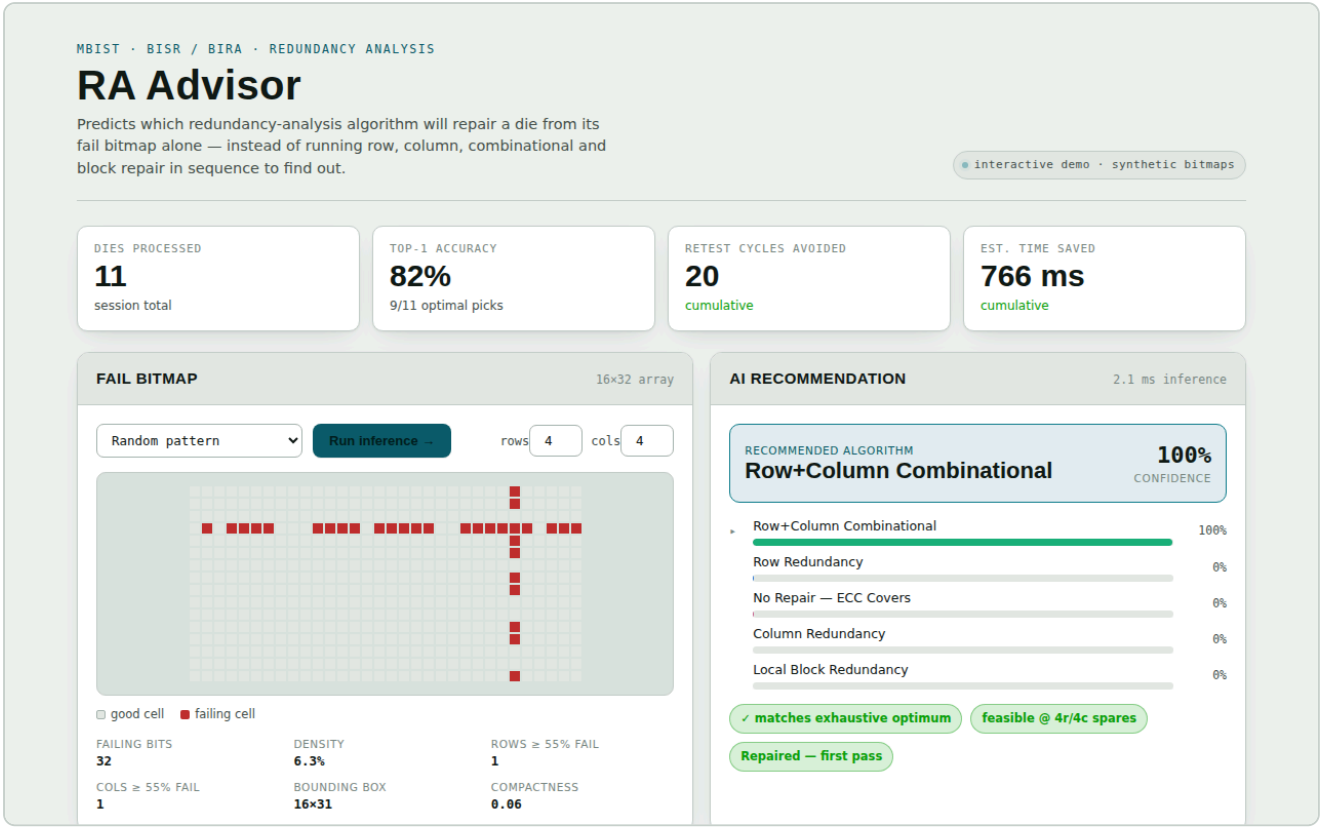

Figure 1. Left: the generated fail bitmap (16×32 array) with extracted features below it. Right: the recommended algorithm, its confidence, the full ranked list of alternatives, and feasibility/outcome verdicts against the configured spare budget.

Each run generates one of six fail-pattern archetypes (scattered single-bit, row-dominant, column-dominant, localized cluster, mixed row+column, or near-clean) so the model's behavior across the full space of realistic defect shapes is visible in one session rather than inferred from a single example.

5.2 Scoring Function

The stand-in "model" scores each of the five algorithm classes from the engineered features (`rowsAbove`, `colsAbove` — counts of rows/columns crossing a 55% fail threshold; `maxRowFrac`, `maxColFrac` — fill fraction of the busiest row/column; `density`; and `compactness` — failing cells divided by bounding-box area), then converts scores to a probability distribution with softmax:

```
score(row)   = 3·rowsAbove + 2·maxRowFrac − 2·colsAbove − 4·maxColFrac
score(col)    = 3·colsAbove + 2·maxColFrac − 2·rowsAbove − 4·maxRowFrac
score(combo) = 3·[rowsAbove>0 && colsAbove>0] + 1.5·min(rowsAbove,colsAbove) +
4·min(maxRowFrac,maxColFrac) + 4·density
score(cluster) = compactness·(5 if blobby else 1) + 3·[blobby] − 2·rowsAbove −
2·colsAbove
score(none)  = 6−total  if total≤2,  else max(0, 1−0.3·total)
```

This is deliberately simple and fully transparent — the point of the reference build is to make the decision logic auditable, not to claim state-of-the-art accuracy. A small amount of random jitter is added before the softmax to emulate the imperfect confidence of a real trained model rather than a hand-tuned rule engine that is always exactly right when its threshold conditions are met.

5.3 Feasibility Engine

Independently of what the model recommends, the dashboard computes the true optimal label the way a production labeling pipeline should: it checks, for each of the five classes, whether applying it would fully cover every failing cell within the configured spare-row and spare-column budget, and if more than one class is feasible, picks the one that consumes the fewest spares. This is the exact mechanism described as the labeling pitfall in Section 4.3, made concrete and runnable. Comparing the model's pick against this feasibility-checked optimum is what produces the "matches exhaustive optimum" verdict and the top-1 accuracy statistic shown in the dashboard.

5.4 Legacy vs. AI-Assisted Timing Model

For every die, the dashboard computes two timelines: the legacy flow tries algorithms in a fixed order (row → column → combinational → cluster → none) until the feasibility-checked correct one is reached, paying a full repair-compute pass each time; the AI-assisted flow pays one small inference cost plus a single repair pass when its recommendation is feasible, or one inference cost plus two repair passes (one failed attempt, one fallback to the correct algorithm) when it isn't.

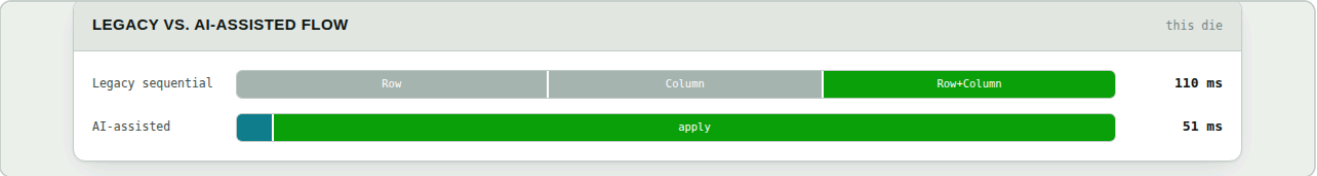

Figure 2. One die's repair timeline under the legacy sequential flow (tries row, then column, then row+column combinational before succeeding — 110 ms) versus the AI-assisted flow (recommends row+column combinational directly — 51 ms).

06 Demonstration Results

> Figures and statistics below are one representative 11-die session against synthetic bitmaps — illustrative of the workflow's shape, not a measurement of any fab's real yield or repair distribution.

METRIC	VALUE	INTERPRETATION
Dies processed	11	Session length for this demonstration run.
Top-1 accuracy	82% (9/11)	Share of dies where the model's first recommendation matched the feasibility-checked optimum.
Retest cycles avoided	20	Repair-algorithm attempts saved versus the fixed try-order, cumulative.
Estimated time saved	766 ms	Cumulative difference between legacy and AI-assisted timelines across all 11 dies.

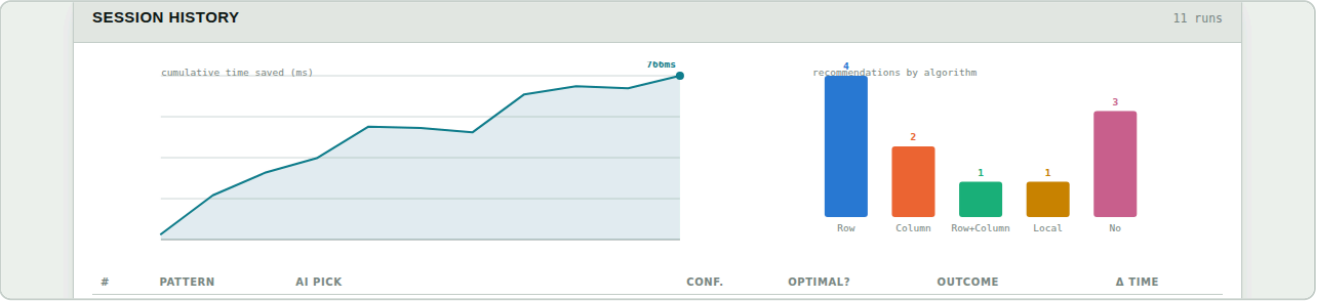

Figure 3. Left: cumulative time saved trending upward across the session (occasional dips correspond to dies where the model's pick required a fallback pass). Right: distribution of recommended algorithm across dies processed — useful for spotting a model that over-favors one class.

The two mismatches in this session were both cases where the model's recommendation was still feasible and repaired the die — just not the spare-minimal choice — or required one fallback pass; none produced a scrapped die. That distinction matters operationally: a wrong-but-feasible recommendation costs some spare-budget efficiency, while a wrong-and-infeasible one costs a full retest cycle, and both are worth tracking separately once this moves beyond a demonstration.

07 Production Implementation Roadmap

Moving from this reference build to a production deployment is a six-step path:

1. **Capture labeled fail bitmaps.**
   Pull historical fail bitmaps from BISR/ATE logs, each tagged with which RA algorithm actually succeeded and how many spares it consumed. Without existing history, run a shadow period where every die tries all algorithms so the winner can be labeled directly.

2. **Engineer the features — or don't.**
   Start with the tabular features from Section 4.1 feeding a gradient-boosted tree; move to a CNN over the raw bitmap only if engineered features demonstrably under-fit real failure modes.

3. **Train as multiclass classification.**
   Label using the feasibility-aware method from Section 4.3 — the algorithm that both repairs the die and minimizes spare usage. Split any holdout set by wafer or lot, not by individual die: dies on the same wafer share defect modes, and a random per-die split leaks signal across the split.

4. **Validate in shadow mode.**
   Run the model alongside the existing brute-force RA flow without acting on its output. Track top-1 accuracy and the feasibility-miss rate — the same two figures this report's Section 6 shows — across several lots before cutover.

5. **Deploy with an automatic fallback.**
   Embed inference in the ATE software layer immediately after MBIST fail capture, or in the post-processing repair database for engineering triage. Keep the legacy sequential path as an automatic fallback whenever the model's recommendation turns out infeasible — this is what keeps a wrong prediction from ever becoming a scrapped die that a correct one would have saved.

6. **Monitor and retrain.**
   New memory IP, a new process node, or a shifted defect mix will drift accuracy over time. Track it per lot and per fab the way Section 6's dashboard does, and retrain on a fixed cadence or when the feasibility-miss rate crosses an agreed threshold.

08 Recommendations and Next Steps

- Start with a shadow-mode data collection pass on one product line before any modeling work — the feasibility-aware labeling method in Section 4.3 depends on having real spare-budget and outcome data, not just fail bitmaps.
- Track top-1 accuracy and feasibility-miss rate as two separate numbers from day one; conflating them hides the operationally important failure mode (a scrapped die) inside

the less costly one (a spare-suboptimal repair).

- Treat the legacy sequential flow as a permanent fallback, not a transitional one — it is the safety net that makes an imperfect model deployable at 82% (or any) top-1 accuracy without increasing scrap rate.
- Revisit the CNN-over-raw-bitmap option only after tabular-feature accuracy plateaus below target on real data; it is strictly more expensive to build, deploy, and maintain.

R References

1. [1] "A Deep Learning Model for Redundancy Analysis Algorithm Recommendation," IEEE Xplore, 2021. ieeexplore.ieee.org/document/9691578
2. [2] "The Key Role of AI in Semiconductor Testing," Siemens EDA Thought Leadership blog. blogs.sw.siemens.com/thought-leadership/the-key-role-of-ai-in-semiconductor-testing
3. [3] "AI/ML for Yield Learning and Test Optimization in Semiconductor Manufacturing: A Review of Adaptive Testing, Smarter Limits, and Diagnostic Preservation," Research Square, 2025. researchsquare.com/article/rs-9464431/v1
4. [4] "AI Accelerator Testing Depends On DFT Innovations," SemiEngineering. semiengineering.com/ai-accelerator-testing-depends-on-dft-innovations
5. [5] "Memory Testing: MBIST, BIRA & BISR — An Insight into Algorithms and Self Repair Mechanism," eInfochips. einfochips.com/blog/memory-testing-an-insight-into-algorithms-and-self-repair-mechanism
6. [6] "Memory Built-In Self-Test (MBIST): Advanced Techniques," ASPDAC 2025 tutorial. aspdac.com/aspdac2025/archive/pdf/T3-1.pdf

The RA Advisor interactive dashboard referenced throughout this report is available as a companion artifact for hands-on exploration of the scoring function, feasibility engine, and session-history metrics described in Sections 5–6.